AI-Generated Smart Contract Security Audit Report



By Savant.Chat

self-transfer-mint.zip

Table of contents

1. Executive Summary 3
2. Vulnerability Statistics 3
3. Findings 3
3.1. Stale Balance Read on Self-Transfer in `SuperToken::transfer_internal` Allows Arbitrary Token Minting 3
4. Disclaimer 5

1. Executive Summary

Audited 1 file for sha256:1ac7a7cf499131bfaa96022a848e60adc79a463706687833290fb5ad780124cb and found 1 finding.

2. Vulnerability Statistics

- **Total findings: 1**

Severity Distribution

Severity	Count
critical	1
high	0
medium	0
minor	0
informational	0
theoretical	0

3. Findings

3.1. Stale Balance Read on Self-Transfer in `SuperToken::transfer_internal` Allows Arbitrary Token Minting

File: self-transfer-mint.rs

Severity: Critical

```
        .extend_ttl(&key, BALANCE_LIFETIME_THRESHOLD, BALANCE_BUMP_AMOUNT);
}

fn transfer_internal(env: &Env, from: &Address, to: &Address, value: i128) {
    require_nonnegative(value);

    let balance_from = read_balance(env, from);
    let balance_to = read_balance(env, to);
    if balance_from < value {
        panic!("insufficient balance");
    }

    write_balance(env, from, balance_from - value);
    write_balance(env, to, balance_to + value);
```

Synopsis

In `self-transfer-mint.rs`, `transfer_internal` reads both the sender (`from`) and recipient (`to`) balances from persistent storage before writing either updated balance back. When `from == to`, the second write (`balance_to + value`) overwrites the first write (`balance_from - value`) using the pre-debit balance, inflating the account's balance by `value` without increasing `total_supply`.

Technical Details

The internal helper `transfer_internal` (lines `84–102` of `self-transfer-mint.rs`) implements token transfers for both `SuperToken::transfer` (line `169`) and `SuperToken::transfer_from` (line `226`).

```
fn transfer_internal(env: &Env, from: &Address, to: &Address, value: i128) {
    require_nonnegative(value);

    let balance_from = read_balance(env, from);
    let balance_to = read_balance(env, to);
    if balance_from < value {
        panic!("insufficient balance");
    }

    write_balance(env, from, balance_from - value);
    write_balance(env, to, balance_to + value);
    ...
}
```

When a transfer is executed where `from` and `to` refer to the same `Address`:

1. `balance_from` is read from `DataKey::Balance(from)` at line 87 (e.g., `B`).
2. `balance_to` is read from `DataKey::Balance(to)` at line 88. Because `from == to` and no write has occurred yet, `balance_to` also equals `B`.
3. Line 89 verifies `B >= value`.
4. Line 93 writes `B - value` to `DataKey::Balance(from)`.
5. Line 94 writes `balance_to + value` (`B + value`) to `DataKey::Balance(to)`, which is the exact same storage key `DataKey::Balance(from)`.

As a result, the debit at line 93 is completely overwritten by the credit at line 94. Instead of leaving the caller's balance unchanged at `B`, the self-transfer increases the balance to `B + value` while `DataKey::TotalSupply` in instance storage remains unchanged.

Proof of Concept

Verdict: true positive proved by PoC
Proven claim: A self-transfer via `SuperToken::transfer` or `transfer_from` on `from == to` leaves the holder's balance at `balance + value` instead of `balance` (because `transfer_internal` overwrites the debit with a stale `balance_to + value` write) while `total_supply` stays fixed, and the effect is repeatable.
PoC artifact: tests/poc_self_transfer_mint.rs (in artifacts.zip)
Evidence: exit code 0

Assumptions and Preconditions

1. An account holds a positive token balance (`balance > 0`), or a spender holds a positive allowance for an account with a positive token balance.
2. The caller invokes `transfer(from, to, value)` (authorizing `from`) or `transfer_from(spender, from, to, value)` (authorizing `spender`) with `from == to` and `0 < value <= balance`.

Verification Checklist

1. Inspect `transfer_internal` in `self-transfer-mint.rs` (lines 84–102).
2. Verify that `SuperToken::transfer` (lines 167–171) and `SuperToken::transfer_from` (lines 197–228) do not restrict `from != to` before calling `transfer_internal`.
3. Confirm that `let balance_from = read_balance(env, from);` (line 87) and `let balance_to = read_balance(env, to);` (line 88) both execute prior to `write_balance(env, from, balance_from - value);` (line 93).
4. Confirm that `write_balance` (lines 76–82) keys persistent storage solely by `DataKey::Balance(addr.clone())`, meaning `from == to` targets the identical persistent storage key in both `write_balance` calls.

Severity Assessment

Level: Critical

I've fully verified the mechanism. Let me summarize my analysis.

Verification performed (independent of the report):

1. **Code path** (`src/lib.rs:84–102`): `transfer_internal` reads `balance_from` (line 87) and `balance_to` (line 88) into locals *before* writing either. Line 93 writes `balance_from - value`; line 94 writes `balance_to + value` to the *same* `DataKey::Balance(from)` key when `from == to`. The credit overwrites the debit → balance becomes `B + value` while `DataKey::TotalSupply` (instance storage, line 121) is never touched.

2. **PoC reproduced**: 1000 → 2000 with `total_supply` fixed at 1000.

3. **Unbounded, not one-shot**: I ran 20 consecutive self-transfers of the full balance → 1,048,576,000 (doubling each time); 62 doublings → 4.6e21. `total_supply` stayed at 1000 the whole time. The attacker needs only a balance of 1 token and one transaction per doubling, so the condition is trivially attacker-arrangeable — no privileged role, no timing, no special configuration.

4. **`transfer_from` path** (line 226 → same helper): with `from == to`, a spender's allowance is decremented while the victim's balance *increases* — a second reachable route into the same helper.

5. **No repair path**: the contract has no `burn`, no `mint`, no admin adjustment, and `TotalSupply` is only written in `__constructor`. The broken invariant is permanent and the inflated balance is a normal, fully transferable balance.

Side observation (does not change the rating): `read_balance` calls `extend_ttl` on the balance key, and the host errors with `Storage/MissingValue` for a key that does not exist (`prepare_extend_ttl → try_get_host_ledger_entry → ok_or_else(MissingValue)`), so `balance_of`/`transfer` to a never-funded address reverts. This is a separate defect; it does not constrain the mint, since the attacker inflates *their own* balance (which necessarily has an entry) and can move it to any address that already holds one.

Severity reasoning: The defect is not style or a trusted-role choice — `transfer` only requires the holder's own `require_auth()`. The consequence is unauthorized minting of the protocol's own token: unlimited, repeatable, and permanent, with `total_supply` left reporting a false figure. Any holder of 1 token can print arbitrary amounts and pass them to counterparties that accept them at face value, destroying the value of every other holder's position and any accounting built on `total_supply`. The rubric places "unauthorized minting" in the critical tier, and every element of that tier is satisfied here: the attacker arranges all preconditions, the magnitude is unbounded, the loss falls on ordinary users' principal positions rather than on the attacker's own funds, and there is no administrative recovery. The missing `extend_ttl`-on-missing-key behavior and the absence of an internal redemption venue mean the theft happens at the token's acceptance points rather than inside this contract, but that only affects the *venue*, not the fact that unbacked tokens are created without limit and cannot be revoked.

Severity: **critical**.

Remediation

Read the recipient's balance only after writing the sender's debited balance (or handle `from == to` explicitly), and use checked arithmetic when crediting the recipient.

```
fn transfer_internal(env: &Env, from: &Address, to: &Address, value: i128) {
    require_nonnegative(value);

    let balance_from = read_balance(env, from);
    if balance_from < value {
        panic!("insufficient balance");
    }
    write_balance(env, from, balance_from - value);

    let balance_to = read_balance(env, to);
    write_balance(
        env,
        to,
        balance_to.checked_add(value).expect("balance overflow"),
    );

    Transfer {
        from: from.clone(),
        to: to.clone(),
        value,
    }
    .publish(env);
}
```

4. Disclaimer

This report records automated audit findings for the reviewed source. It does not guarantee that the source is free of defects or security vulnerabilities.